

Exhaustive Line-by-Line Code Breakdown: analyzer.py

Complete Word-by-Word Analysis of Detective Agent Suspect Evaluation Engine (Lines 1 to 131 — Zero Gaps)

SECTION 1: IMPORTS & CLASS INITIALIZATION (LINES 1–13)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
1	<code>from typing import Dict, Any, List</code>	Imports type hint constructs `Dict`, `Any`, and `List` from Python's standard `typing` module for static type annotations.
2	<code>from rag.retriever import CaseRetriever</code>	Imports the custom `CaseRetriever` class from the `rag.retriever` package module to interact with vector database searches.
3	<code></code>	Blank spacing line separating import statements from class definition.
4	<code></code>	Second padding blank line for PEP 8 compliance.
5	<code>class DetectiveAgent:</code>	Declares the main `DetectiveAgent` class encapsulating suspect evaluation and RAG retrieval logic.
6	<code></code>	Blank line after class header.
7	<code>def __init__(self):</code>	Defines the constructor initializer method `__init__` taking the standard instance reference `self`.
8	<code># Create the RAG retriever when the agent is initialized.</code>	Comment explaining that the instance initializes a vector retriever dependency upon creation.
9	<code>self.retriever = CaseRetriever()</code>	Instantiates `CaseRetriever` and attaches it to the instance variable `self.retriever`.
10	<code></code>	Blank line separating methods.
11	<code>def reload_cases(self) -> int:</code>	Defines public method `reload_cases` returning an integer count of newly reloaded cases.
12	<code>"""Refresh indexed cases after a new JSON file is uploaded."""</code>	Docstring describing how the method refreshes vector index records after new data uploads.
13	<code>return self.retriever.reload()</code>	Delegates execution to `self.retriever.reload()` and returns the resulting integer count.

SECTION 2: SUSPECT METHOD SIGNATURE & QUERY SYNTHESIS (LINES 14–33)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
14	<code></code>	Blank line separating methods.
15	<code>def evaluate_suspect(</code>	Declares the primary evaluation method `evaluate_suspect`.
16	<code>self,</code>	Instance parameter reference `self`.
17	<code>name: str,</code>	Parameter `name`: string representing the suspect's name.
18	<code>behavior: str,</code>	Parameter `behavior`: string describing observed actions/behavior.
19	<code>mo_suspected: str,</code>	Parameter `mo_suspected`: string indicating suspected modus operandi.
20	<code>personality_notes: str</code>	Parameter `personality_notes`: string containing psychological or observational notes.
21	<code>) -> Dict[str, Any]:</code>	Closing argument signature specifying return type as a dictionary with string keys and arbitrary values.
22	<code></code>	Blank line.
23	<code># Build one query while avoiding duplicate text from the frontend.</code>	Comment explaining query construction and deduplication strategy for search queries.
24	<code>query_parts = []</code>	Initializes empty list `query_parts` to accumulate non-empty, unique text segments.
25	<code>for value in (behavior, mo_suspected, personality_notes):</code>	Iterates through a tuple containing input behavioral descriptors.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
26	<code>value_text = str(value or "").strip()</code>	Safely casts parameter to string (handling `None`), trimming leading/trailing whitespace.
27	<code>if value_text and value_text not in query_parts:</code>	Check ensuring trimmed text is non-empty and not already present in `query_parts`.
28	<code>query_parts.append(value_text)</code>	Appends unique text segment to the `query_parts` list.
29	<code>query_str = "; ".join(query_parts)</code>	Joins accumulated unique text segments into a single semi-colon separated search string.
30		Blank line.
31	<code># Search the RAG case database for the most similar historical cases.</code>	Comment introducing the RAG retrieval phase.
32	<code>retrieved_cases = self.retriever.search_similar_cases(</code>	Calls `search_similar_cases` on retriever instance with constructed query.
33	<code>query_str,</code>	First positional argument: synthesized string `query_str`.

SECTION 3: RAG RETRIEVAL & DISTANCE THRESHOLD FILTERING (LINES 34–52)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
34	<code>top_k=3</code>	Keyword argument restricting search results to top 3 nearest historical cases.
35	<code>)</code>	Closes function call for `search_similar_cases`.
36		Blank line.
37	<code># CaseRetriever returns distance = 1 - cosine similarity.</code>	Comment explaining mathematical relationship between distance metric and cosine similarity.
38	<code># Smaller distance means stronger similarity.</code>	Comment clarifying distance interpretation rule.
39	<code>MATCH_DISTANCE_THRESHOLD = 0.80</code>	Defines constant threshold: distance must be <= 0.80 (similarity >= 0.20) to qualify as a match.
40		Blank line.
41	<code>matched_cases: List[Dict[str, Any]] = []</code>	Initializes empty typed list to hold filtered matching case dictionaries.
42	<code>if retrieved_cases:</code>	Evaluates if retrieval call yielded any results.
43	<code>for case in retrieved_cases:</code>	Iterates through each retrieved case record dictionary.
44	<code>dist = case.get("distance")</code>	Extracts `distance` key from case dictionary safely.
45	<code>if dist is not None and float(dist) <= MATCH_DISTANCE_THRESHOLD:</code>	Validates distance presence and checks if float value meets threshold.
46	<code>matched_cases.append(case)</code>	Appends qualifying case record to `matched_cases` list.
47		Blank line.
48	<code># Dynamic Scoring Logic</code>	Comment marking start of risk scoring logic block.
49	<code># Use actual similarity instead of only counting the number of matches.</code>	Comment detailing continuous similarity scoring model approach.
50	<code>base_score = 15</code>	Establishes baseline propensity/risk score of 15 points.
51		Blank line.
52	<code>if matched_cases:</code>	Branches logic if vector match filtering produced at least one valid case.

SECTION 4: VECTOR SIMILARITY MATH & QUALITY CLASSIFICATION (LINES 53–82)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
53	<code>similarities = [</code>	Starts list comprehension to extract individual numerical similarity scores.
54	<code>float(case.get("similarity", 0.0))</code>	Safely parses `similarity` value to float, defaulting to 0.0 if missing.
55	<code>for case in matched_cases</code>	Iterates through matching case items.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
56	<code>]</code>	Closes similarity list comprehension.
57		Blank line.
58	<code>strongest_similarity = max(similarities, default=0.0)</code>	Calculates maximum similarity score across all matches.
59	<code>average_similarity = (</code>	Starts computation for mean similarity score.
60	<code>sum(similarities) / len(similarities)</code>	Divides sum of similarity scores by total match count.
61	<code>if similarities</code>	Ternary check preventing division by zero.
62	<code>else 0.0</code>	Default value if similarities list is empty.
63	<code>)</code>	Closes average similarity assignment.
64		Blank line.
65	<code>similarity_score = (</code>	Computes weighted composite score component.
66	<code>(strongest_similarity * 60)</code>	Applies 60% weight multiplier to peak similarity metric.
67	<code>+ (average_similarity * 25)</code>	Applies 25% weight multiplier to average similarity metric.
68	<code>)</code>	Closes weighted calculation.
69		Blank line.
70	<code>score = round(</code>	Rounds final clamped score to nearest integer.
71	<code>min(95, max(15, base_score + similarity_score))</code>	Clamps score within range [15, 95] added to baseline.
72	<code>)</code>	Closes rounding expression.
73		Blank line.
74	<code>strongest_similarity = max(similarities, default=0.0)</code>	Re-evaluates max similarity metric for quality tier classification.
75	<code>if strongest_similarity >= 0.55:</code>	Evaluates if peak similarity meets strong match threshold (≥ 0.55).
76	<code>match_quality = "Strong case-index similarity"</code>	Sets label to strong match descriptor.
77	<code>elif strongest_similarity >= 0.35:</code>	Evaluates if peak similarity meets moderate match threshold (≥ 0.35).
78	<code>match_quality = "Moderate case-index similarity"</code>	Sets label to moderate match descriptor.
79	<code>else:</code>	Fallback branch for weaker vector matches (< 0.35).
80	<code>match_quality = "Weak case-index similarity"</code>	Sets label to weak match descriptor.
81		Blank line.
82	<code>else:</code>	Else branch executed when no vector match passes distance threshold.

SECTION 5: KEYWORD FALLBACK HEURISTICS & RISK CATEGORIZATION (LINES 83–115)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
83	<code># Fallback heuristic: analyze keyword severity if no vector match is found.</code>	Comment detailing fallback strategy when semantic search yields no matches.
84	<code>combined_text = (</code>	Constructs lowercased unified corpus from input fields.
85	<code>f"{behavior} {mo_suspected} {personality_notes}"</code>	Interpolates all behavioral parameters into string.
86	<code>).lower()</code>	Converts full string to lowercase for case-insensitive keyword scanning.
87		Blank line.
88	<code>severe_keywords = [</code>	Defines list of critical criminal keyword indicators.
89	<code>"murder", "kill", "weapon", "assault", "robbery",</code>	List items: violent crime keywords.
90	<code>"theft", "break-in", "crime", "stolen", "force", "threat"</code>	List items: property and threat crime keywords.
91	<code>]</code>	Closes keyword list definition.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
92		Blank line.
93	<code>kw_matches = sum(</code>	Calculates total occurrences of listed keywords in combined text.
94	<code>1 for kw in severe_keywords</code>	Generator yielding 1 per keyword found in text.
95	<code>if kw in combined_text</code>	Condition checking keyword presence inside combined text string.
96	<code>)</code>	Closes sum computation.
97		Blank line.
98	<code>if kw_matches > 0:</code>	Evaluates if any severe keywords were detected.
99	<code>score = min(</code>	Clamps heuristic score upper boundary.
100	<code>95,</code>	Maximum allowable score cap.
101	<code>max(</code>	Clamps heuristic score lower boundary.
102	<code>25,</code>	Minimum score threshold when keywords are present.
103	<code>base_score + (kw_matches * 15)</code>	Adds 15 points per matched severe keyword to base score.
104	<code>)</code>	Closes `max` function call.
105	<code>)</code>	Closes `min` function call.
106	<code>else:</code>	Fallback branch when no keywords and no vector matches exist.
107	<code>score = 15</code>	Assigns minimum baseline score of 15.
108	<code>match_quality = "No reliable case-index match"</code>	Sets match quality label indicating absence of vector evidence.
109		Blank line.
110	<code># Convert the numerical score into a risk category.</code>	Comment introducing numerical-to-categorical conversion.
111	<code>if score >= 70:</code>	Evaluates high risk threshold (>= 70%).
112	<code>risk_level = "HIGH RISK"</code>	Assigns "HIGH RISK" categorical status.
113	<code>elif score >= 40:</code>	Evaluates medium risk threshold (>= 40%).
114	<code>risk_level = "MEDIUM RISK"</code>	Assigns "MEDIUM RISK" categorical status.
115	<code>else:</code>	Branch for scores below 40%.

SECTION 6: RESPONSE PACKAGING & OUTPUT STRUCTURE (LINES 116–131)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
116	<code>risk_level = "LOW RISK"</code>	Assigns "LOW RISK" categorical status.
117		Blank line.
118	<code>return {</code>	Returns structured response dictionary containing evaluation metrics.
119	<code>"suspect_name": name,</code>	KeyValue: suspect's name string.
120	<code>"tendency_score": f"{score}%",</code>	KeyValue: formatted numerical score with percentage sign.
121	<code>"risk_level": risk_level,</code>	KeyValue: categorical risk level assessment.
122	<code>"match_quality": match_quality,</code>	KeyValue: descriptive case match quality string.
123	<code>"summary": (</code>	KeyValue: dynamically constructed natural language summary string start.
124	<code>f"Suspect pattern aligns with {len(matched_cases)}</code> <code>"</code>	Summary text string interpolating count of matched historical cases.
125	<code>f"historical cases in the vector database."</code>	Summary text string continuation for successful vector match.
126	<code>if matched_cases else</code>	Ternary condition checking if vector matches were found.
127	<code>"No direct vector match in database. Risk evaluated</code> <code>from behavioral indicators."</code>	Fallback text string when no vector match exists.
128	<code>),</code>	Closes summary string assignment tuple.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
129	<code>"similar_cases": matched_cases</code>	KeyValue: list of matching historical case dictionaries.
130	<code>}</code>	Closes dictionary return statement.
131	<code></code>	Final empty newline ending `analyzer.py`.